

✓ HANDS - ON: PROMPT ENGINEERING Y SISTEMAS RAG

Una vez vista la masterclass *Prompt Engineering y Sistemas RAG*, se proporciona el siguiente **Colab** para construir, en vivo, distintas estrategias de prompting y un mini sistema de RAG.

Usamos **Groq** para tener acceso a inferencia con GPU gratuita, sin necesidad de infraestructura propia.

De igual manera, se proporciona la **solución** de este notebook a través del siguiente [enlace](#).

✓ CONFIGURACIÓN DEL ENTORNO

Para no exponer tu **API key** directamente en el código, Colab ofrece un panel de **Secrets** (ícono de llave en la barra lateral izquierda) donde puedes guardarla de forma segura, añadiendo un nombre asociado a la **API key** para guardarla dentro de una variable y usarla dentro del notebook.

```
# Instalar cliente de Groq y leer API key desde Colab Secrets
!pip install -q groq sentence-transformers scikit-learn

# Importar librerías
from google.colab import userdata
from groq import Groq
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Leer API key desde Colab Secrets
api_key = userdata.get("GROQ_API_KEY")

# Crear cliente de Groq
client = Groq(api_key=api_key)

# Cargar modelo para embeddings
modelo_embeddings = SentenceTransformer("all-MiniLM-L6-v2")

print("Entorno configurado correctamente.")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate l
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please se
modules.json: 100% 349/349 [00:00<00:00, 15.7kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 3.70kB/s]

README.md: 100% 10.5k/10.5k [00:00<00:00, 326kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 2.31kB/s]

config.json: 100% 612/612 [00:00<00:00, 19.3kB/s]

model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.59MB/s

model.safetensors: downloading bytes: 85.0MB, 7.79MB/s

Loading weights: 100% 103/103 [00:00<00:00, 1209.42it/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 8.02kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 3.91MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 8.32MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 2.05kB/s]

config.json: 100% 190/190 [00:00<00:00, 12.9kB/s]

Entorno configurado correctamente.
```

➤ ESTRATEGIAS DE PROMPTS

↳ 10 cells hidden

✓ RAG: BUSCAR ANTES DE RESPONDER

RAG separa el proceso en dos pasos: primero un sistema de búsqueda encuentra los fragmentos más relevantes de una base de conocimiento propia (usando *embeddings* y similitud semántica); después, esos fragmentos se le entregan al modelo junto con la pregunta para generar la respuesta.

```
# Instalar sentence-transformers
!pip install -q sentence-transformers
```

```
# Definir la base de conocimiento (política de devoluciones) y generar sus embeddings
from sentence_transformers import SentenceTransformer

# Modelo para generar embeddings
modelo = SentenceTransformer("all-MiniLM-L6-v2")

# Base de conocimiento: política de devoluciones
documentos = [
    "Los clientes pueden solicitar la devolución de un producto dentro de los primeros 30 días posteriores a",
    "El producto debe encontrarse en buenas condiciones y conservar sus accesorios originales.",
    "Los productos defectuosos pueden devolverse dentro de los primeros 90 días. En estos casos, se puede sol
]

# Generar embeddings de los documentos
embeddings = modelo.encode(documentos)

print("Documentos:", len(documentos))
print("Forma de los embeddings:", embeddings.shape)
```

Loading weights: 100%

103/103 [00:00<00:00, 1833.95it/s]

Documentos: 3

Forma de los embeddings: (3, 384)

```
# Definir una función que calcule la similitud entre la pregunta y cada fragmento, y regrese el más relevante
from sklearn.metrics.pairwise import cosine_similarity
```

```
def buscar_fragmento(pregunta):
    # Generar el embedding de la pregunta
    embedding_pregunta = modelo.encode([pregunta])

    # Calcular la similitud coseno entre la pregunta y cada documento
    similitudes = cosine_similarity(embedding_pregunta, embeddings)[0]

    # Encontrar el fragmento con mayor similitud
    indice_mas_relevante = similitudes.argmax()

    # Regresar el fragmento más relevante
    return documentos[indice_mas_relevante]
```

```
# Enviar la pregunta junto con el fragmento recuperado al modelo y mostrar la respuesta con RAG
# Pregunta del usuario
pregunta = "¿Cuánto tiempo tengo para devolver un producto defectuoso?"
```

```
# Recuperar el fragmento más relevante
fragmento = buscar_fragmento(pregunta)
```

```
# Construir el prompt con el contexto recuperado
prompt = f"""
Responde la siguiente pregunta utilizando únicamente la información del contexto.
```

```
Contexto:
{fragmento}
```

```
Pregunta:
{pregunta}
```

```
Si la respuesta no está contenida en el contexto, indica que no hay información suficiente.
"""
```

```
# Enviar la pregunta y el contexto al modelo
respuesta = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": prompt}
    ]
)
```

```
# Mostrar respuesta
print("Fragmento recuperado:")
print(fragmento)
```

```
print("\nRespuesta con RAG:")
print(respuesta.choices[0].message.content)
```

Fragmento recuperado:

Los productos defectuosos pueden devolverse dentro de los primeros 90 días. En estos casos, se puede solicitar

Respuesta con RAG:

Tienes hasta 90 días para devolver un producto defectuoso.

✓ CHALLENGE: ASISTENTE DE POLÍTICAS CON RAG

Una vez visto el ***Hands-On: Prompt Engineering y Sistemas RAG***, se presenta el siguiente reto para que el alumnado pueda repasar y reforzar lo aprendido dentro de la clase.

Se construirá un pequeño asistente con RAG sobre un documento propio, comparando la respuesta **con RAG** contra la respuesta **sin RAG** para la misma pregunta. En esta solución se usa como base de conocimiento el propio reglamento de evaluación del curso IA Aplicada con Llama.

IMPORTANTE: Para su revisión, **es indispensable que los apartados anteriores se encuentren llenados con el código visto durante la sesión.**

✓ INSTRUCCIONES:

1. Define tu base de conocimiento y genera embeddings:

- Lee la API key previamente configurada desde **Colab Secrets** e instala/importa las librerías necesarias.
- Construye una lista llamada `documentos` con 3 fragmentos de un documento real de tu propio contexto (reglamento, políticas, FAQs, etc.) y genera sus embeddings con `sentence-transformers`.

```
# Leer API key, instalar e importar librerías

!pip install -q sentence-transformers scikit-learn groq

from google.colab import userdata
from groq import Groq
from sentence_transformers import SentenceTransformer

#Leer API key desde Colab Secrets
api_key = userdata.get("GROQ_API_KEY")

#Crear cliente de Groq
client = Groq(api_key=api_key)

#Cargar modelo para generar embeddings
modelo = SentenceTransformer("all-MiniLM-L6-V2")

print("Entorno configurado correctamente")
```

```

modules.json: 100% 349/349 [00:00<00:00, 7.77kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 2.02kB/s]
README.md: 100% 10.5k/10.5k [00:00<00:00, 212kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 1.58kB/s]
config.json: 100% 612/612 [00:00<00:00, 10.4kB/s]
model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.87MB/s
model.safetensors: downloading bytes: 85.0MB, 8.13MB/s
Loading weights: 100% 103/103 [00:00<00:00, 2582.24it/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 10.4kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 4.35MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 13.7MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 3.47kB/s]
config.json: 100% 190/190 [00:00<00:00, 11.8kB/s]
Entorno configurado correctamente

```

```
# Definir la lista documentos y generar sus embeddings
```

```

documentos = [
    """
    Las pruebas de validación de diseño (DVT) tienen como objetivo verificar
    que el producto cumple con los requisitos técnicos y ambientales establecidos.
    Estas pruebas pueden incluir vibración, ciclos térmicos, humedad y pruebas
    de aislamiento eléctrico.
    """,

    """
    Durante la validación de producción (PVT), se verifica que el proceso de
    manufactura pueda producir el producto de manera persistente. Esta etapa
    puede incluir estudios de MSA, Gage R&R y validación de los equipos de prueba.
    """,

    """
    Las pruebas de fin de línea (EOL) se realizan durante producción para
    verificación que cada unidad fabricada cumpla con los criterios funcionales
    establecidos antes de ser liberados. Los resultados deben registrarse para
    mantener la trazabilidad del producto.
    """]
#Genera embeddings
embeddings = modelo.encode(documentos)

print("Número de documentos:", len(documentos))
print("Dimensiones de los embeddings:", embeddings.shape)

```

```

Número de documentos: 3
Dimensiones de los embeddings: (3, 384)

```

2. Recupera el fragmento relevante: Define una función `buscar_fragmento(pregunta)` que calcule la similitud coseno y regrese el fragmento más relevante para una pregunta dada.

```

# Definir la función buscar_fragmento

from sklearn.metrics.pairwise import cosine_similarity

def buscar_fragmento(pregunta):
    #Generar el embedding de la pregunta

```

```

embedding_pregunta = modelo.encode([pregunta])

#Calcular similitud coseno entre la pregunta y los documentos
similitudes = cosine_similarity(
    embedding_pregunta,
    embeddings
)[0]

#obtener el indice del fragmento más relvante
indice_mas_relevante = similitudes.argmax()

#Regresar el fragmento más relevante
return documentos[indice_mas_relevante]

pregunta = "¿Qué se verifica durante la validación de producción?"

fragmento = buscar_fragmento(pregunta)

print("Fragmento más relevante:")
print(fragmento)

```

Fragmento más relevante:

Durante la validación de producción (PVT), se verifica que el proceso de manufactura pueda producir el producto de manera persistente. Esta etapa puede incluir estudios de MSA, Gage R&R y validación de los equipos de prueba.

3. Genera la respuesta sin RAG: Envía una pregunta real sobre tu documento directamente al modelo (sin ningún fragmento de contexto) y guarda la respuesta en `respuesta_sin_rag`.

```

# Consultar la pregunta sin RAG y guardar el resultado en respuesta_sin_rag
pregunta = "¿Qué se verifica durante la validación de producción?"

respuesta = client.chat.completions.create(
    model = "openai/gpt-oss-20b",
    messages = [
        {
            "role": "user",
            "content": pregunta
        }
    ]
)

respuesta_sin_rag = respuesta.choices[0].message.content

print("Respuesta sin RAG:")
print(respuesta_sin_rag)

```

```

**Rendimiento y escalabilidad** | • Tiempo de respuesta bajo carga típica <br>• Utilización de CPU, memoria
**Seguridad** | • Parches y actualizaciones de seguridad instalados <br>• Configuración de firewall, reglas
**Resiliencia y recuperación** | • Back-ups automáticos y restauración <br>• Pruebas de failover (alta disp
**Monitoreo y alertas** | • Sistema de logging (centralizado, nivel de detalle adecuado) <br>• Métricas de
**Compliance / Documentación** | • Registro de auditoría (cambios de configuración, despliegues) <br>• Manu
**Integraciones externas** | • APIs de terceros, servicios de mensajería, pagos, etc. <br>• Pruebas de con
**Pruebas de carga de usuarios reales** | • Simulación de usuarios simultáneos <br>• Pruebas de stress <br>

```

```

- Realizar pruebas de rendimiento con herramientas como JMeter, Gatling o k6.
- Ejecutar pruebas de seguridad (OWASP ZAP, Burp, etc.) y auditoría de logs.

3. **Revisión y aprobación**
- Analizar resultados, revisar métricas y logs.
- Validar que todas las métricas clave (tiempo de respuesta, disponibilidad, error rate) están dentro de lo esperado.
- Obtener aprobación formal del equipo de QA, DevOps y del Product Owner.

4. **Despliegue en producción**
- Una vez aprobada la validación, proceder a mover la versión final a producción con un plan de despliegue.
- Monitorizar continuamente los indicadores clave (KPI) y estar preparados para revertir si algo sale mal.

---

### ¿Qué pasa si no se verifica algo crítico?

- **Errores en la producción**: Pueden implicar caídas, pérdida de datos o brechas de seguridad.
- **Revisiones manuales**: Se vuelve más caro y lleva más tiempo corregir problemas post-despliegue.
- **Sanciones regulatorias**: En entornos regulados (farmacéutica, banca, salud) la falta de validación puede tener consecuencias legales.
- **Pérdida de reputación**: Los usuarios finales pueden perder confianza y migrar a competidores.

---

### Resumen rápido

Durante la validación de producción se verifica:

1. **Entorno y configuración** (servidores, variables, paquetes).
2. **Datos** (integridad y migraciones).
3. **Funcionalidad** (flujos críticos, UI/API).
4. **Rendimiento** (tiempos, carga, escalabilidad).
5. **Seguridad** (parches, reglas, pruebas).
6. **Resiliencia** (back-ups, failover).
7. **Monitoreo** (logs, alertas).
8. **Compliance** (documentación, normativas).
9. **Integraciones** (APIs externas, mensajería).
10. **Cargas reales** (usuarios concurrentes, estrés).

Al cubrir estos puntos, te aseguras de que el sistema no solo está técnicamente funcional, sino también operativo y seguro.

```

4. Genera la respuesta con RAG: Recupera el fragmento relevante con tu función y envía la pregunta junto con ese fragmento al modelo. Guarda la respuesta en `respuesta_con_rag`.

```

# Consultar la pregunta con RAG y guardar el resultado en respuesta_con_rag

#Recuperar el fragmento más relevante
fragmento_relevante = buscar_fragmento(pregunta)

#Construir el prompt con el texto recuperado
prompt_rag = f"""
Responder la pregunta utilizando únicamente la información proporcionada en el contexto.

Contexto:
{fragmento_relevante}

Pregunta:
{pregunta}

Si la respuesta no se encuentra en el contexto, indica que no hay información suficiente.
"""

#Enviar la pregunta junto con el fragmento recuperado al modelo
respuesta = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {
            "role": "user",
            "content": prompt_rag
        }
    ]
)

```

```

    ]
)

#Guardar la respuesta en la variable solicitada
respuesta_con_rag = respuesta.choices[0].message.content

print("Fragmento recuperado")
print(fragmento_relevante)

print("\nRespuesta con RAG:")
print(respuesta_con_rag)

```

```

**Rendimiento y escalabilidad** | • Tiempo de respuesta bajo carga típica <br>• Utilización de CPU, memoria
**Seguridad** | • Parches y actualizaciones de seguridad instalados <br>• Configuración de firewall, reglas
**Resiliencia y recuperación** | • Back-ups automáticos y restauración <br>• Pruebas de failover (alta disponibilidad)
**Monitoreo y alertas** | • Sistema de logging (centralizado, nivel de detalle adecuado) <br>• Métricas de
**Compliance / Documentación** | • Registro de auditoría (cambios de configuración, despliegues) <br>• Mant
**Integraciones externas** | • APIs de terceros, servicios de mensajería, pagos, etc. <br>• Pruebas de con
**Pruebas de carga de usuarios reales** | • Simulación de usuarios simultáneos <br>• Pruebas de stress <br>

```

Flujo típico de validación de producción

- **Preparación****
 - Replicar el entorno de producción (infraestructura, bases de datos, datos de prueba) lo más fielmente posible
 - Asegurar que todas las dependencias externas (bases de datos, servicios de terceros) estén accesibles.
- **Ejecución de pruebas****
 - Automatizar pruebas de regresión y de extremo a extremo.
 - Realizar pruebas de rendimiento con herramientas como JMeter, Gatling o k6.
 - Ejecutar pruebas de seguridad (OWASP ZAP, Burp, etc.) y auditoría de logs.
- **Revisión y aprobación****
 - Analizar resultados, revisar métricas y logs.
 - Validar que todas las métricas clave (tiempo de respuesta, disponibilidad, error rate) están dentro de límites aceptables.
 - Obtener aprobación formal del equipo de QA, DevOps y del Product Owner.
- **Despliegue en producción****
 - Una vez aprobada la validación, proceder a mover la versión final a producción con un plan de despliegue
 - Monitorizar continuamente los indicadores clave (KPI) y estar preparados para revertir si algo sale mal

¿Qué pasa si no se verifica algo crítico?

- ****Errores en la producción****: Pueden implicar caídas, pérdida de datos o brechas de seguridad.
- ****Revisiones manuales****: Se vuelve más caro y lleva más tiempo corregir problemas post-despliegue.
- ****Sanciones regulatorias****: En entornos regulados (farmacéutica, banca, salud) la falta de validación puede tener consecuencias legales.
- ****Pérdida de reputación****: Los usuarios finales pueden perder confianza y migrar a competidores.

Resumen rápido

Durante la validación de producción se verifica:

- **Entorno y configuración**** (servidores, variables, paquetes).
- **Datos**** (integridad y migraciones).
- **Funcionalidad**** (flujos críticos, UI/API).
- **Rendimiento**** (tiempos, carga, escalabilidad).
- **Seguridad**** (parches, reglas, pruebas).
- **Resiliencia**** (back-ups, failover).
- **Monitoreo**** (logs, alertas).
- **Compliance**** (documentación, normativas).
- **Integraciones**** (APIs externas, mensajería).
- **Cargas reales**** (usuarios concurrentes, estrés).

Al cubrir estos puntos, te aseguras de que el sistema no solo está técnicamente funcional, sino también opera

5. Compara y concluye: Imprime ambas respuestas y concluye cuál de las dos evitó mejor una alucinación o dio una

```
# Mostrar ambas respuestas para comparar
```

```
print("=" * 60)
print("RESPUESTA SIN RAG")
print("=" * 60)
print(respuesta_sin_rag)

print("=" * 60)
print("RESPUESTA CON RAG")
print("=" * 60)
print(respuesta_con_rag)
```

```

| **Rendimiento y escalabilidad** | • Tiempo de respuesta bajo carga típica <br>• Utilización de CPU, memoria
| **Seguridad** | • Parches y actualizaciones de seguridad instalados <br>• Configuración de firewall, reglas
| **Resiliencia y recuperación** | • Back-ups automáticos y restauración <br>• Pruebas de failover (alta disponibilidad)
| **Monitoreo y alertas** | • Sistema de logging (centralizado, nivel de detalle adecuado) <br>• Métricas de
| **Compliance / Documentación** | • Registro de auditoría (cambios de configuración, despliegues) <br>• Manuales
| **Integraciones externas** | • APIs de terceros, servicios de mensajería, pagos, etc. <br>• Pruebas de conectividad
| **Pruebas de carga de usuarios reales** | • Simulación de usuarios simultáneos <br>• Pruebas de stress <br>

```

```
---
```

```
### Flujo típico de validación de producción
```

1. ****Preparación****
 - Replicar el entorno de producción (infraestructura, bases de datos, datos de prueba) lo más fielmente posible.
 - Asegurar que todas las dependencias externas (bases de datos, servicios de terceros) estén accesibles.
2. ****Ejecución de pruebas****
 - Automatizar pruebas de regresión y de extremo a extremo.
 - Realizar pruebas de rendimiento con herramientas como JMeter, Gatling o k6.
 - Ejecutar pruebas de seguridad (OWASP ZAP, Burp, etc.) y auditoría de logs.
3. ****Revisión y aprobación****
 - Analizar resultados, revisar métricas y logs.
 - Validar que todas las métricas clave (tiempo de respuesta, disponibilidad, error rate) están dentro de los límites aceptables.
 - Obtener aprobación formal del equipo de QA, DevOps y del Product Owner.